Notebook setup

Run the next cell first. In Google Colab it clones the course repository, then imports the libraries and the course modules. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Import or generate the in silico dataset

At the beginning of each notebook you decide whether to import the pre-generated experiments or to generate your own dataset with the in silico process model. For details on the process simulator, see the notebook `00_Process_Introduction.ipynb`.

- To import the dataset set `DATASET_SOURCE = "import"`.
- To generate a new dataset set `DATASET_SOURCE = "generate"`. The files are written to the folder `generated/`.

When running the notebook for the first time, we recommend the "import" option.

```
""" Select dataset source, one of (import, generate) """
DATASET_SOURCE = "import"
```

```
if DATASET_SOURCE == "import":
    # Define filename to import
    FILENAME = "owu.csv"
    FILEPATH = "datasets/"
    # Import OWU and DOE train data
    doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
    owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
    owu.index =
pd.MultiIndex.from_product([list(range(int(len(owu)/15))),list(range(15))], names=
["run","time"])
    # Import OWU Test data
    FILENAME = "owu_test.csv"
    doe_test =
pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None, usecols =
["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
    owu_test = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD",
"X:Glc", "X:Lac", "X:Titer","W:Feed"])
    owu_test.index =
pd.MultiIndex.from_product([list(range(int(len(owu_test)/15))),list(range(15))],
names=["run","time"])
```

```
if DATASET_SOURCE == "generate":
    # DOE Dataset definition (variable = [lower bound, upper bound])
    # Model parameters: Dictate the behaviour of the cell process
    MU_G_MAX = 0.05;
    MU_D_MAX = 0.025;
    K_G_GLC  = 1;
    K_I_LAC  = 30;
    K_D_LAC  = 50;
    K_GLC    = 0.04;
    K_LAC    = 0.06;
    K_PROD   = 1;
    # Process parameters: Conditions at which process is run
    FEED_START = [1, 4]
    FEED_END = [8, 12]
    GLC_FEED_RATE = [5, 20]
    GLC_0 = [10, 80.0]
    VCD_0 = [0.1, 1.0]
    # Number of experiments to generate
    NUM_RUNS = 40
    # Filename and filepath for the dataset
    FILENAME = "owu.csv"
    FILEPATH = "generated/"
    # Collect parameters to dictionary
    VAR_LIMS = {"mu_g_max":MU_G_MAX, "mu_d_max": MU_D_MAX, "K_g_Glc" : K_G_GLC,
"K_I_Lac" : K_I_LAC, "K_d_Lac" : K_D_LAC, "k_Glc" : K_GLC, "k_Lac" : K_LAC, "k_Prod"
: K_PROD,
        "feed_start" : FEED_START,"feed_end" : FEED_END, "Glc_feed_rate" :
GLC_FEED_RATE, "Glc_0" : GLC_0, "VCD_0" : VCD_0}
    # Generate Dataset
    data = simulator.generate_data(VAR_LIMS, NUM_RUNS, FILEPATH + FILENAME)
    # Import DOE
    doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
    # Import OWU
    owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
    owu.index = pd.MultiIndex.from_product([list(range(NUM_RUNS)),list(range(15))],
names=["run","time"])
```

Regression modelling

In this section we first import the train and test experiments generated with the process simulator (see `00_Process_Introduction.ipynb`). Alternatively you can generate your own training data with the "generate" option.

These experimental data are used to build models with the aim of predicting the final titer.

Two types of models are considered:

- *Initial conditions model*: takes only the process conditions and initial conditions (the DOE) as inputs. It is a black-box model that disregards the process behaviour and considers only the outcome.
- *Historical model*: takes the full process history as input. This model needs to handle high-dimensional and collinear data.

Initial conditions models

From the OWU train and test datasets we create the BWU data, the DOE data and the target data. The DOE data contain the initial process conditions (the Z variables) of the experiments. The target data contain only the final titer at day 14 (or alternatively the aggregates).

```
# Train set
bwu = simulator.generate_bwu(owu)
tar = simulator.generate_y(bwu,return_aggr=False)
# Test set
bwu_test = simulator.generate_bwu(owu_test)
tar_test = simulator.generate_y(bwu_test,return_aggr=False)
```

```
# Visualize input data and output data
dataset = copy.copy(doe) # inputs
dataset["Target"] = tar # outputs
dataset
```

	feed_start	feed_end	Glc_feed_rate	Glc_0	VCD_0	Target
0	2.500000	10.000000	12.500000	45.000000	0.550000	1565.310032
1	1.948980	10.244898	18.622449	40.714286	0.274490	854.449233
2	3.295918	9.346939	14.642857	17.857143	0.127551	1206.096655
3	3.479592	9.020408	15.561224	47.857143	0.843878	1587.937536
4	1.459184	9.591837	7.295918	33.571429	0.421429	811.55299
...	...	...	...	...	...	...
45	2.377551	8.612245	6.989796	59.285714	0.513265	1139.068175
46	2.683673	8.775510	10.357143	57.857143	0.201020	1424.48449
47	3.173469	10.000000	8.520408	63.571429	0.696939	1464.164972
48	2.561224	10.081633	6.071429	39.285714	0.715306	639.543576
49	1.826531	10.816327	14.948980	42.142857	0.347959	1297.031632

50 rows × 6 columns

Multiple linear regression

We first fit a linear regression model for the final titer using the data loaded above.

We then characterize the model with the regression summary (ANOVA table), which reports the importance of each coefficient and of the overall model.

Create a linear model with preprocessing

We create a linear model that predicts the final titer from the initial conditions.

- Input matrix: `doe`. The values of the manipulated process parameters of each experiment.
- Output target: `tar`. The final titer at the end of each experiment (or the aggregates).

The inputs are standardized (mean 0, standard deviation 1) so that the coefficients are comparable.

```python
# Fit Linear Model
datascaler = StandardScaler()
does = datascaler.fit_transform(doe.values)

X = sm.add_constant(does)
y = tar.values
mod = sm.OLS(y.astype(float), X.astype(float))
res = mod.fit()
print(res.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.269
Model:                            OLS   Adj. R-squared:                  0.186
Method:                 Least Squares   F-statistic:                     3.240
Date:                Wed, 28 Oct 2026   Prob (F-statistic):             0.0141
Time:                        04:04:39   Log-Likelihood:                -357.18
No. Observations:                  50   AIC:                             726.4
Df Residuals:                      44   BIC:                             737.8
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       1153.5475     46.179     24.980      0.000    1060.480    1246.615
x1            66.2397     47.334      1.399      0.169     -29.156     161.635
x2           109.1749     47.629      2.292      0.027      13.184     205.166
x3            58.6069     47.344      1.238      0.222     -36.809     154.023
x4            75.0607     47.819      1.570      0.124     -21.313     171.434
x5            72.4612     49.215      1.472      0.148     -26.725     171.647
==============================================================================
Omnibus:                        3.705   Durbin-Watson:                   1.875
Prob(Omnibus):                  0.157   Jarque-Bera (JB):                3.534
Skew:                          -0.604   Prob(JB):                        0.171
Kurtosis:                       2.514   Cond. No.                         1.45
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

The interpretation of the main items in the table:

- R-squared : the coefficient of determination. It is the proportion of the variance in the dependent variable that is predictable/explained
- Adj. R-squared : Adjusted R-squared is the modified form of R-squared adjusted for the number of independent variables in the model. Value of adj. R-squared increases, when we include extra variables which actually improve the model.
- F-statistic : the ratio of the variance explained by the model to the variance of the residuals. It determines the overall significance of the model.
- coef : the coefficients of the independent variables and the constant term in the equation.
- t : the value of the t-statistic, the ratio of the estimated coefficient to its standard error. The corresponding p-value (P>|t|) tells whether the coefficient is significantly different from zero.

Model training and inference

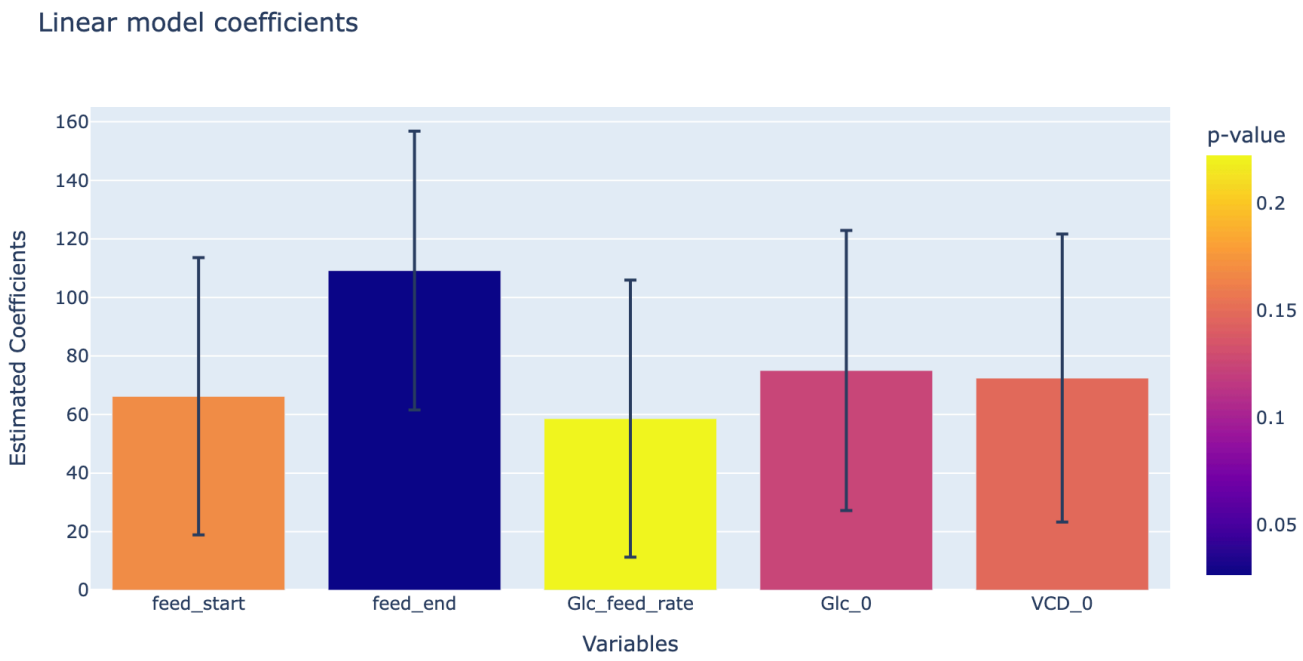
In this section we fit the linear model as a scikit-learn pipeline, which makes it easy to add polynomial and interaction features. We check the predictions of the model against the training data and the test data for the final titer.

- `USE_POLYDEGREE = 1` gives the linear model from above, `2` adds quadratic terms.
- `USE_INTERACTIONSONLY = True` adds only the products between different inputs, not the squares.

```
""" Normalize data """
USE_NORMALIZATION = True
""" Polynomial degree of features """
USE_POLYDEGREE = 1
""" Add only interaction between features"""
USE_INTERACTIONSONLY = True
```

```
# Define pipeline
poly_features = PolynomialFeatures(degree=USE_POLYDEGREE,
interaction_only=USE_INTERACTIONSONLY,include_bias=False)
standard_scaler = StandardScaler(with_mean=USE_NORMALIZATION,
with_std=USE_NORMALIZATION)
lm_model = LinearRegression()
pipe = Pipeline([('features', poly_features),
                 ('scaler', standard_scaler),
                 ('model', lm_model)])
# Fit model
X = doe
y = tar
pipe.fit(X,y)

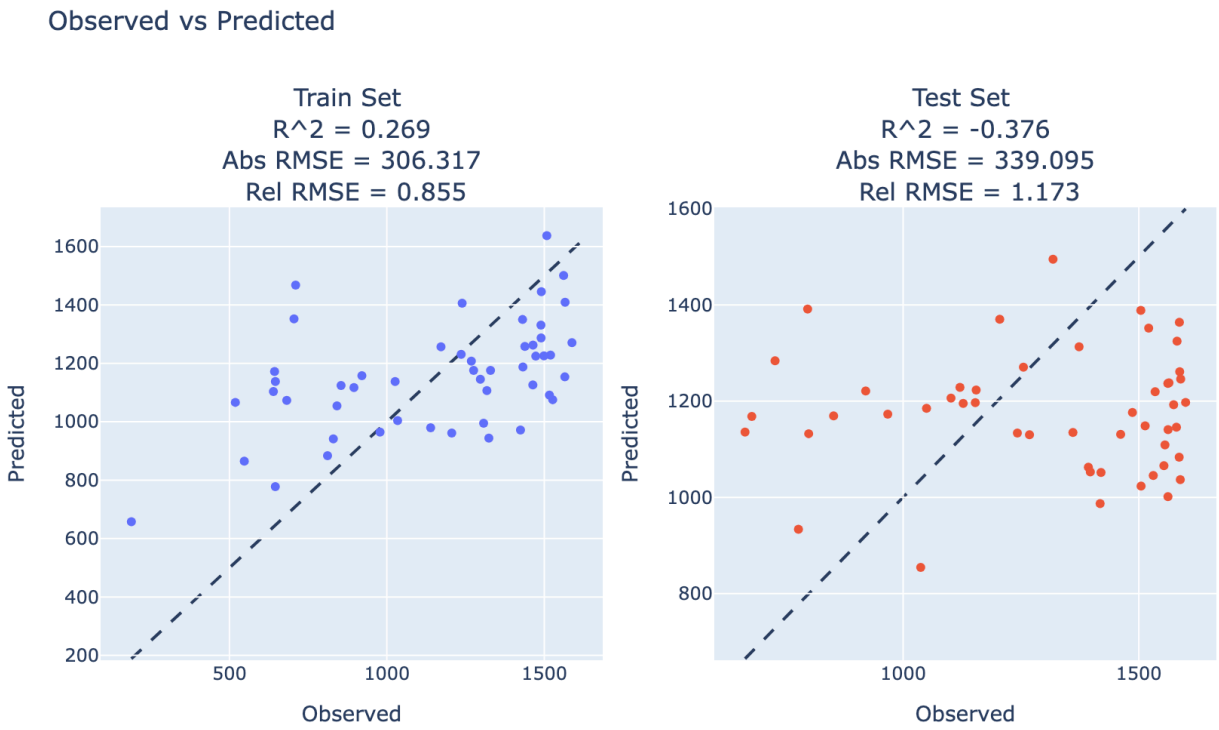
# Model Features and Coefficients
X_columns = poly_features.get_feature_names_out()
lm_coefficients = lm_model.coef_
# Model Std Error and p-values (not via sklearn)
X_preproc = pd.DataFrame(pipe.steps[1][1].transform(pipe.steps[0]
[1].transform(X)),columns=X_columns)
ols_model = sm.OLS(y.values.astype(float),
sm.add_constant(X_preproc).values.astype(float)).fit()
ols_coefficients = ols_model.params[1:]
if X_preproc.shape[1]<X_preproc.shape[0]:
  ols_stderror = ols_model.bse[1:]
  ols_pvalue = ols_model.pvalues[1:].round(4)
else:
  print("\n !!!!! Model is overspecified !!!!! \n Results in perfect fit and
unreliable coefficients  \n")
  ols_stderror = None
  ols_pvalue = None
```



Evaluate the model on the train and test set

The relative RMSE is the RMSE divided by the standard deviation of the observations: a value of 1 means the model is no better than predicting the mean.

```
# Make predictions
yhat = pipe.predict(X)
X_test = doe_test
y_test = tar_test
yhat_test = pipe.predict(X_test)
# Plot observed vs predicted with error metrics (R2, absolute and relative RMSE)
plothelpers.observed_vs_predicted(y, yhat, y_test, yhat_test)
```



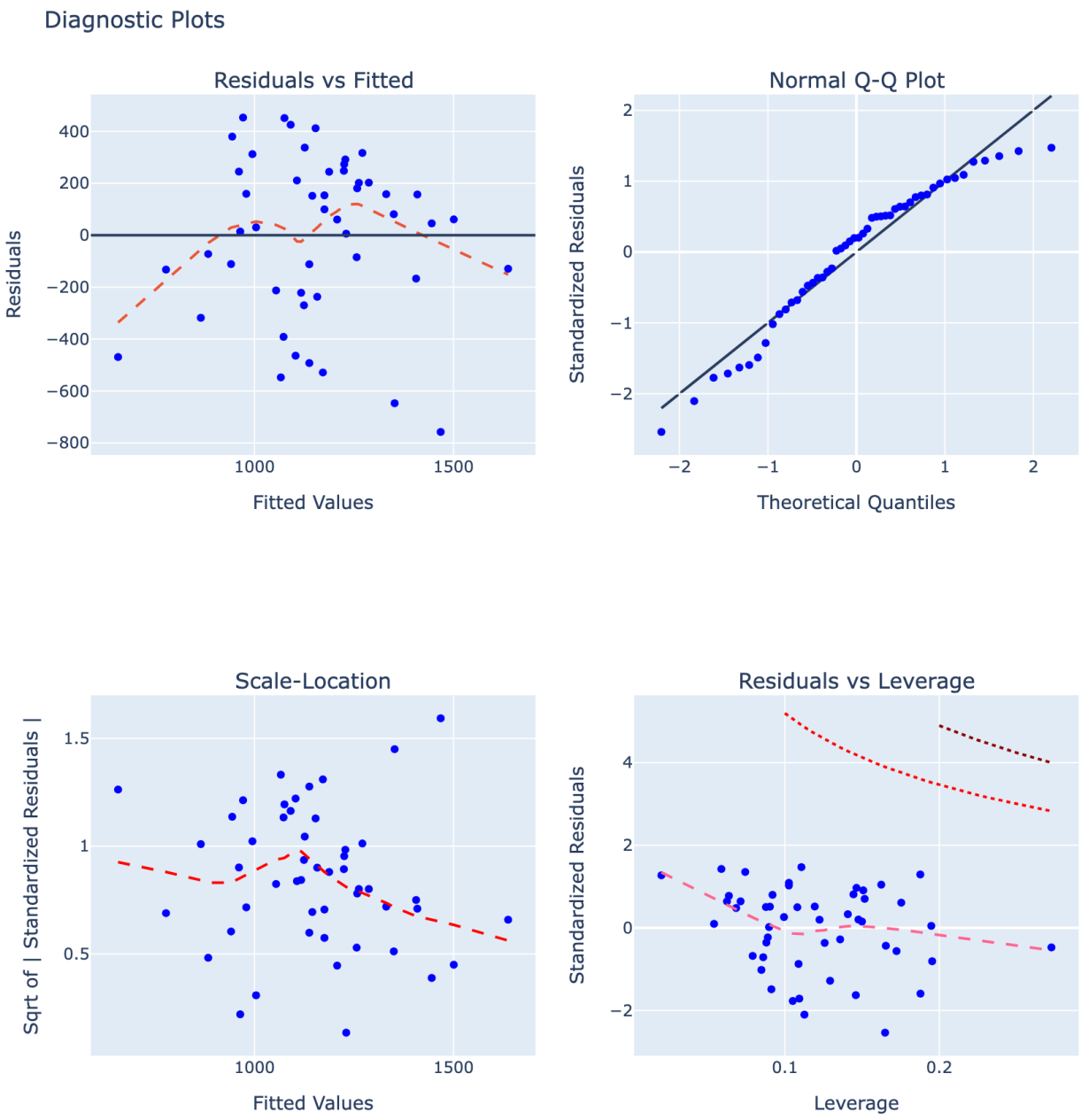
Model diagnostic plots

There are four main assumptions associated with a linear regression model:

- Linearity: the relationship between X and the mean of Y is linear.
- Homoscedasticity: the variance of the residuals is the same for any value of X.
- Independence: the observations are independent of each other.
- Normality: the errors are normally distributed.

These can be verified with so-called diagnostic plots.

```
# Quantities and Statistics necessary for Diagnostic Plots
model_X = pipe.steps[1][1].transform(pipe.steps[0][1].transform(X))
model_y = y.values.reshape(-1)
model_fitted_y = ols_model.fittedvalues
model_residuals = ols_model.resid
model_residuals_trendline = sm.nonparametric.lowess(exog=model_fitted_y,
endog=model_residuals)
model_norm_residuals = ols_model.get_influence().resid_studentized_internal
model_norm_qq = stats.probplot(model_norm_residuals, dist="norm")
model_norm_residuals_abs_sqrt = np.sqrt(np.abs(model_norm_residuals))
model_norm_residuals_abs_sqrt_trendline =
sm.nonparametric.lowess(exog=model_fitted_y, endog=model_norm_residuals_abs_sqrt)
model_abs_resid = np.abs(model_residuals)
model_leverage = ols_model.get_influence().hat_matrix_diag
model_leverage_trendline = sm.nonparametric.lowess(exog=model_leverage,
endog=model_norm_residuals)
```



Interpretation of the diagnostic plots

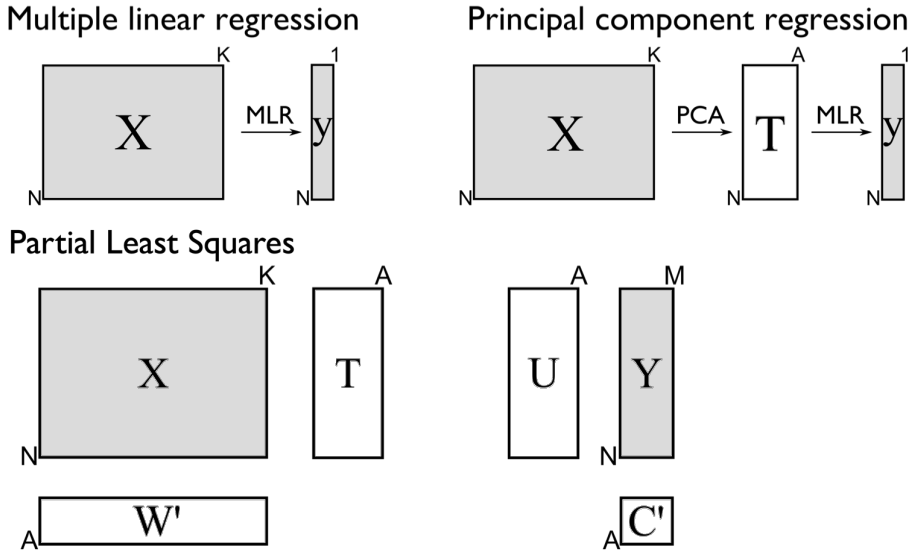
- **Residuals vs Fitted**: This graph shows if there are any nonlinear patterns in the residuals, and thus in the data as well. One of the mathematical assumptions in building an OLS model is that the data can be fit by a line. If this assumption holds and our data can be fit by a linear model, then we should see a relatively flat line when looking at the residuals vs fitted. An ideal Residuals vs Fitted plot will look like random noise; there won't be any apparent patterns in the scatterplot and the red line would be horizontal.

- **Normal Q-Q Plot**: This plot shows if the residuals are normally distributed. A good normal QQ plot has all of the residuals lying on or close to the red line. If there are several points that fall far away from the diagonal line. This is indicative of the errors not being normally distributed. The model can suffer from "heavy tails", "skewness", etc.

- **Scale-Location**: This plot is a way to check if the residuals suffer from non-constant variance, aka heteroscedasticity. The more horizontal the red line is, the more likely the data is homoscedastic.
- **Residuals vs Leverage**: Unlike outliers, which have an unusually large y value, leverage points have extreme x values. This can have damaging effects on the model because the β coefficients are very sensitive to leverage points. Here we can identify problematic observations. Points with a Cook's distance larger than 0.5 are considered influential points.

Model comparison

The figure summarizes how the different regression models of this notebook relate to each other.



PLS model

In this section we keep the same data structure but train a partial least squares (PLS) model. PLS allows us to use more variables/columns without the model being overspecified, because it regresses on a few latent variables instead of the original variables.

For a single response y and k predictors, the PLS regression model with h latent variables is expressed as

$$X = TP^T + E$$

$$y = Tq^T + f$$

where T are the scores, P the X-loadings, q the y-loadings and E, f the residuals. The scores are computed from X with the weights W.

The variable importance in projection (VIP) score summarizes the importance of each variable for the h latent variables. For the j-th variable it is calculated as

$$VIP_j = \sqrt{\frac{k \sum_{a=1}^{h} SS_a \left(w_{aj} / \|w_a\| \right)^2}{\sum_{a=1}^{h} SS_a}}$$

where SS_a is the sum of squares of y explained by latent variable a. Since the average of the squared VIP scores equals 1, the "greater than one" rule is generally used as the criterion for variable selection.

Create a PLS model with a given number of latent variables

Create a PLS model to predict the final titer from the initial conditions.

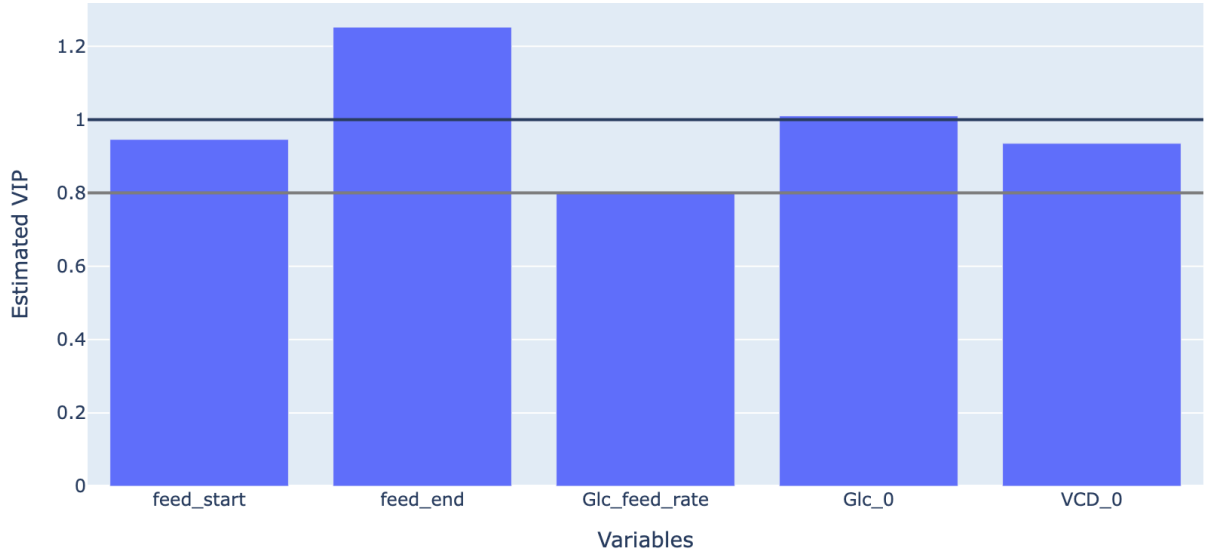
- Input matrix: `doe`. The values of the manipulated process parameters of each experiment.
- Output target: `tar`. The final titer at the end of each experiment (or the aggregates).

Select the number of latent variables of the model (the maximum number of latent variables is equal to the number of columns of the input matrix, 5 without interaction terms).

```
""" Number of latent Variables """
USE_LV = 5
""" Normalize data """
USE_NORMALIZATION = True
""" Polynomial degree of features """
USE_POLYDEGREE = 1
""" Add only interaction between features"""
USE_INTERACTIONSONLY = True
```

```
# Define pipeline
poly_features = PolynomialFeatures(degree=USE_POLYDEGREE,
interaction_only=USE_INTERACTIONSONLY,include_bias=False)
standard_scaler = StandardScaler(with_mean=USE_NORMALIZATION,
with_std=USE_NORMALIZATION)
pls_model = PLSRegression(n_components=USE_LV,scale=False)
pipe = Pipeline([('features', poly_features),
                 ('scaler', standard_scaler),
                 ('model', pls_model)])
# Fit PLS model
X = doe
y = tar
pipe.fit(X,y)
# Model features and variable importance
X_columns = poly_features.get_feature_names_out()
X_preproc = pd.DataFrame(pipe.steps[1][1].transform(pipe.steps[0]
[1].transform(X)),columns=X_columns)
pls_vip = modelhelpers.vip(X_preproc,y,pls_model)
```

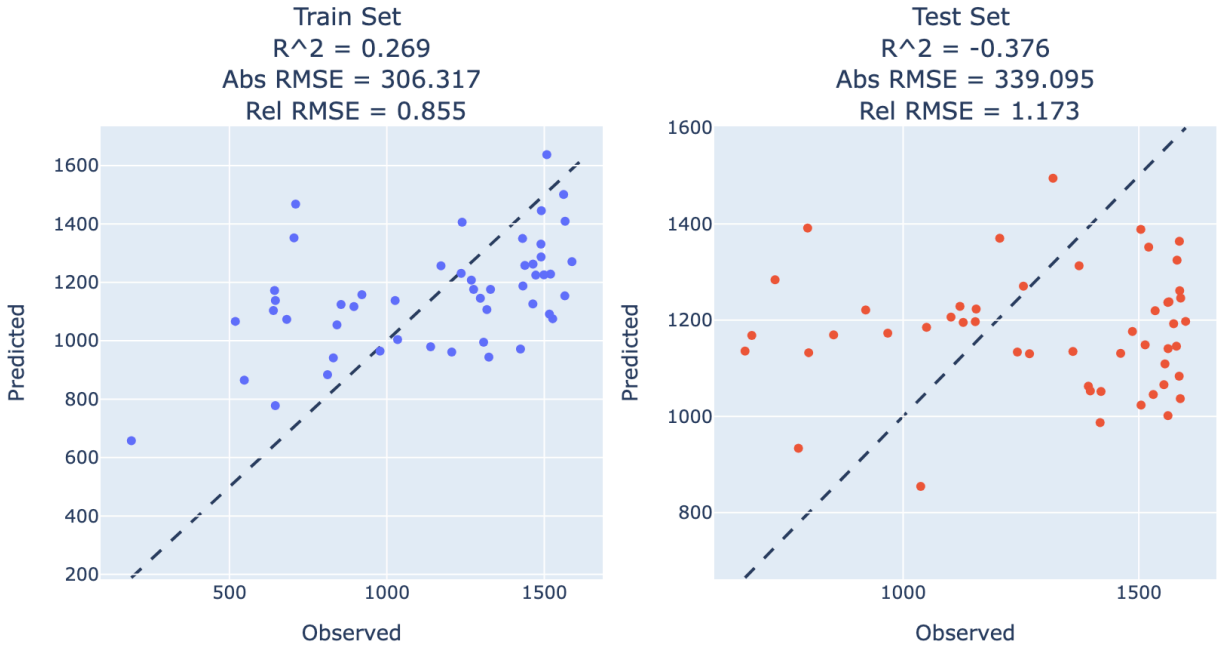
VIP scores of PLS model



Evaluate the model on the train and test set

```
# Make predictions
yhat = pipe.predict(X)
X_test = doe_test
y_test = tar_test
yhat_test = pipe.predict(X_test)
# Plot observed vs predicted with error metrics (R2, absolute and relative RMSE)
plothelpers.observed_vs_predicted(y, yhat, y_test, yhat_test)
```

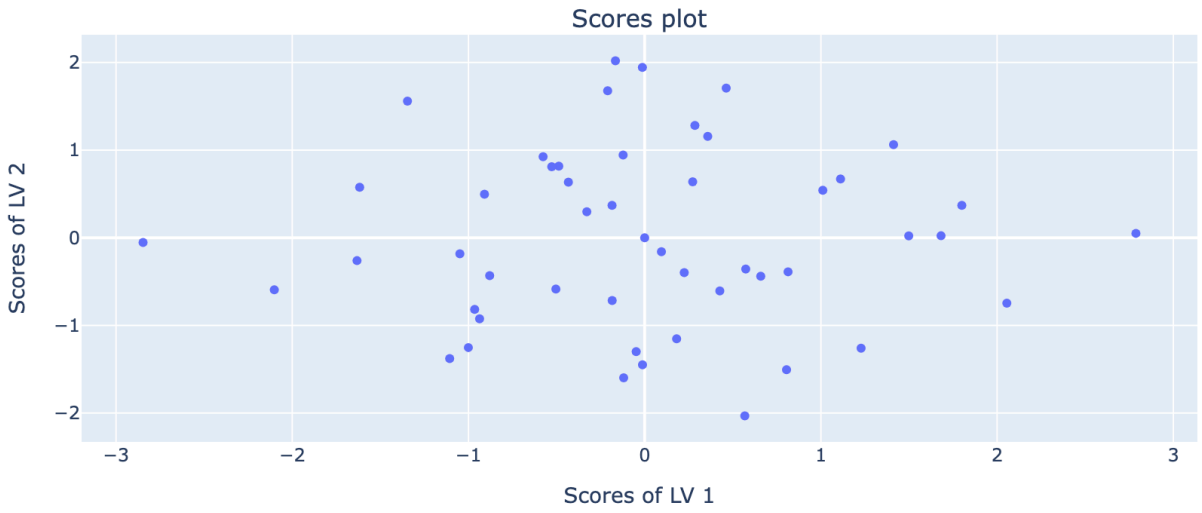
Observed vs Predicted

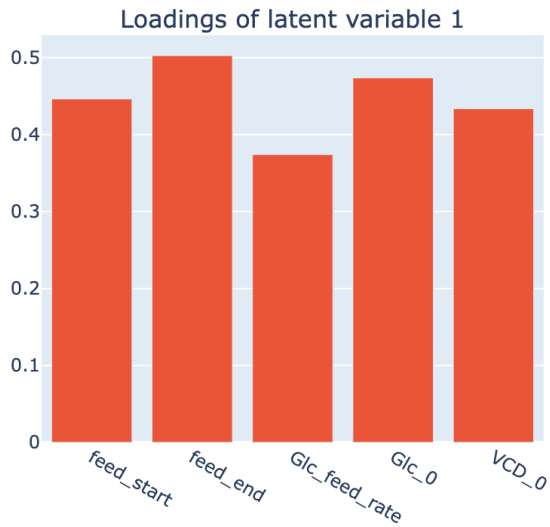


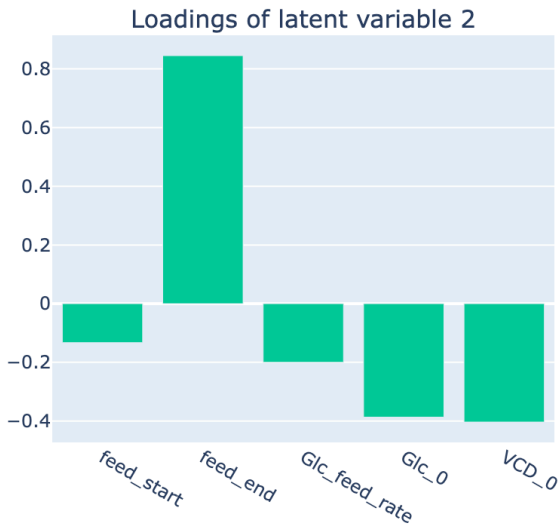
Scores and loadings of the PLS model

The scores show where each experiment lies in the latent variable space, the loadings show how the input variables build each latent variable.

```
""" Latent variable on x-axis and y-axis """
LV_X_AXIS = 1
LV_Y_AXIS = 2
```







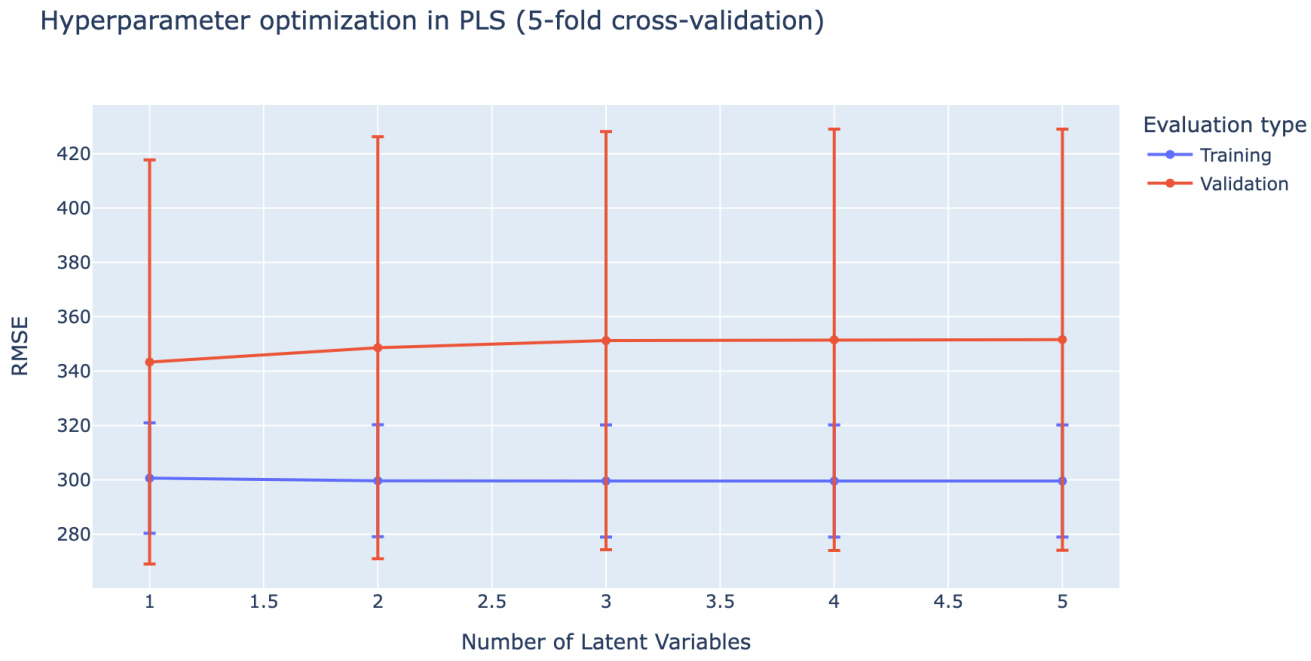
Hyperparameter optimization in cross-validation

In this section we run a typical cross-validation to define the optimal number of latent variables.

N-fold PLS models are trained using (N-1) folds each. Each model is then evaluated on the remaining unused fold. This is repeated for different numbers of latent variables. The number of latent variables

with the lowest validation RMSE is chosen as optimal.

```
range_LV = range(1, X_preproc.shape[1]+1)
train_eval, valid_eval = validation_curve(pipe, X, y, param_name =
"model__n_components", param_range=list(range_LV),scoring =
'neg_root_mean_squared_error')
train_score=-np.mean(train_eval,axis=1)
valid_score=-np.mean(valid_eval,axis=1)
train_std = np.std(train_eval,axis=1)
valid_std = np.std(valid_eval,axis=1)
fig = go.Figure()
fig.add_trace(go.Scatter( x=list(range_LV), y=train_score,
error_y=dict(type='data',array=train_std,visible=True), name="Training"))
fig.add_trace(go.Scatter( x=list(range_LV), y=valid_score,
error_y=dict(type='data',array=valid_std,visible=True), name="Validation"))
fig.update_layout(title="Hyperparameter optimization in PLS (5-fold cross-
validation)",xaxis_title="Number of Latent
Variables",yaxis_title="RMSE",legend_title="Evaluation type")
fig.show()
```



Historical model

In a so-called historical model, the data of the different experiments are ordered into a batch-wise unfolded (BWU) matrix, where every row corresponds to one experiment and the columns are the process variables at every time point.

The BWU matrix can be used to predict final properties of the experiment, like critical quality attributes (CQAs), which are typically the cumulated effect of the whole experiment profile.

In this example we use the BWU matrix to predict the final titer. Clearly, the titer information has to be removed from the BWU matrix.

Multiple linear model

Unlike for the initial conditions model, there are several problems with using a linear regression / response surface model for historical data. The main problem is that MLR requires the dataset X (the BWU data) to have more rows (samples) than columns (variables). A second problem is that if $X^T X$ has full rank only because of noise, the inverse is unstable and small changes in the noise realization produce dramatically different results.

If we look at the shape of our BWU matrix below, it has more variables (columns) than samples (rows), which makes it impossible to fit an ordinary linear model.

```
# (number of runs, number of variables x days)
bwu.shape
```

```
(50, 75)
```

Historical PLS model

We demonstrate that with more information and more variables, PLS becomes a model with good predictions and a low prediction error (RMSEP).

Create the BWU matrix

Here the BWU matrix is created. The values of the manipulated variables are added as the first columns of the matrix. The following pre-processing is performed:

- Remove the titer (and also lactate and the feed) columns
- Remove the days after `USE_DAYS` and the initial day 0
- Eliminate invariant columns
- Add the process parameters (DOE) at the beginning
- Create a PLS model from the resulting matrix to the final titer

```
""" Number of days of process history """
USE_DAYS = 14
""" Remove variables """
REMOVE_VARS = ['Feed','Titer','Lac']
""" Number of latent variables """
N_LV = 5
""" Normalize data """
USE_NORM = True
```

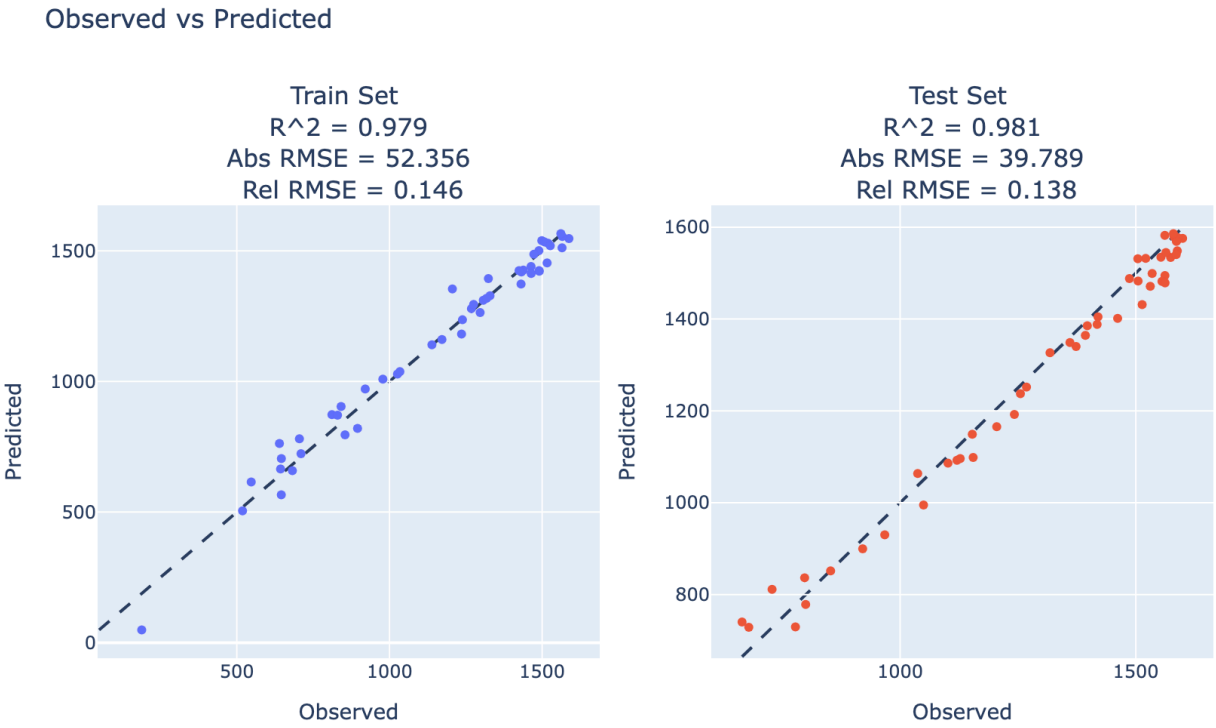
```
# Create and preprocess bwu matrix
X_bwu = bwu
remove_columns = []
# Remove Variables
if 'Titer' in REMOVE_VARS:
    remove_columns.extend([c for c in bwu.columns if c.startswith('X:Titer')])
if 'Lac' in REMOVE_VARS:
    remove_columns.extend([c for c in bwu.columns if c.startswith('X:Lac')])
if 'Feed' in REMOVE_VARS:
    remove_columns.extend([c for c in bwu.columns if c.startswith('W:Feed')])
if 'Glc' in REMOVE_VARS:
    remove_columns.extend([c for c in bwu.columns if c.startswith('X:Glc')])
if 'VCD' in REMOVE_VARS:
    remove_columns.extend([c for c in bwu.columns if c.startswith('X:VCD')])
# Remove History
for d in range(USE_DAYS,15):
    remove_columns.extend([c for c in bwu.columns if c.endswith(':0')])
    remove_columns.extend([c for c in bwu.columns if c.endswith(":"+str(d))])
# Remove Invariant
remove_columns.extend(list(bwu.columns[~(bwu != bwu.iloc[0]).any().values]))
# Add and remove columns
X_preproc = pd.concat([doe, X_bwu.drop(set(remove_columns),axis=1)], axis=1)
print(X_preproc.columns)
```

```
Index(['feed_start', 'feed_end', 'Glc_feed_rate', 'Glc_0', 'VCD_0', 'X:VCD:1',
       'X:VCD:2', 'X:VCD:3', 'X:VCD:4', 'X:VCD:5', 'X:VCD:6', 'X:VCD:7',
       'X:VCD:8', 'X:VCD:9', 'X:VCD:10', 'X:VCD:11', 'X:VCD:12', 'X:VCD:13',
       'X:Glc:1', 'X:Glc:2', 'X:Glc:3', 'X:Glc:4', 'X:Glc:5', 'X:Glc:6',
       'X:Glc:7', 'X:Glc:8', 'X:Glc:9', 'X:Glc:10', 'X:Glc:11', 'X:Glc:12',
       'X:Glc:13'],
      dtype='str')
```

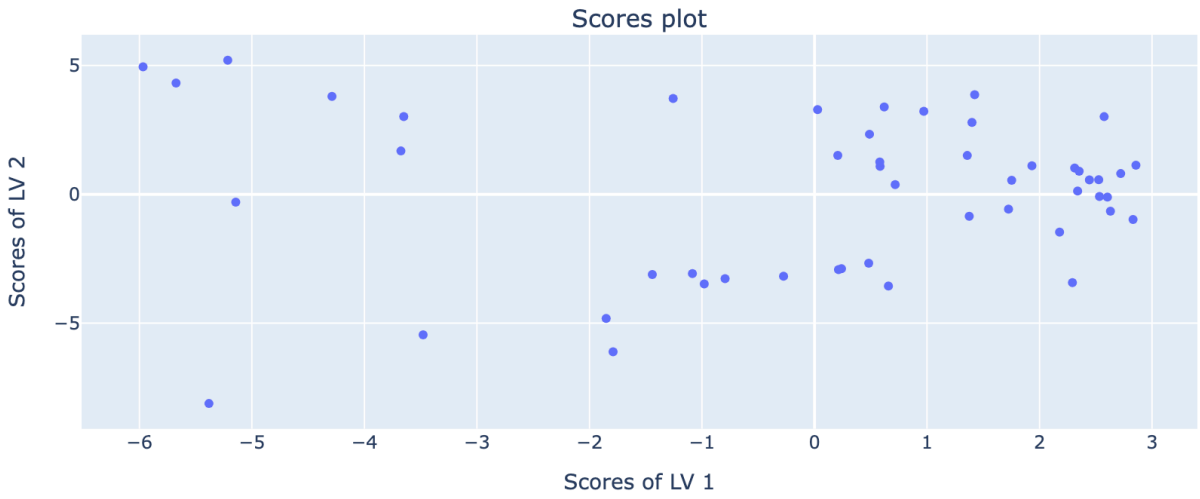
Train and evaluate the model on the train and test set

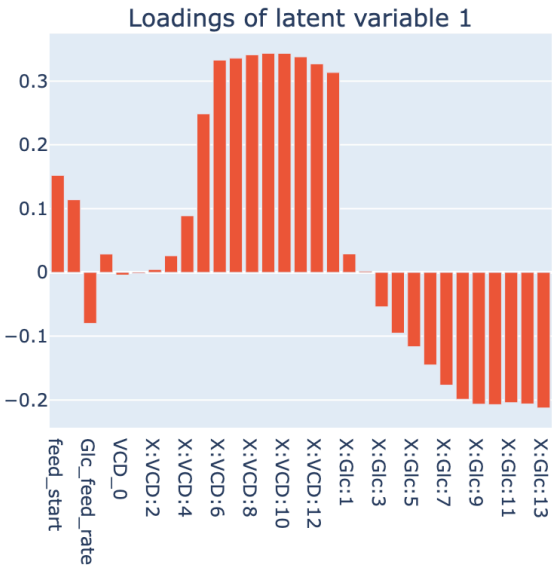
```
# Define Pipeline
pscaler = StandardScaler(with_mean=USE_NORM,with_std=USE_NORM)
pls_bwu = PLSRegression(n_components=N_LV)
pipe = Pipeline([('scaler', pscaler), ('pls', pls_bwu)])
# Train PLS model
y = tar
X = X_preproc
X_columns = X.columns
pipe.fit(X,y)

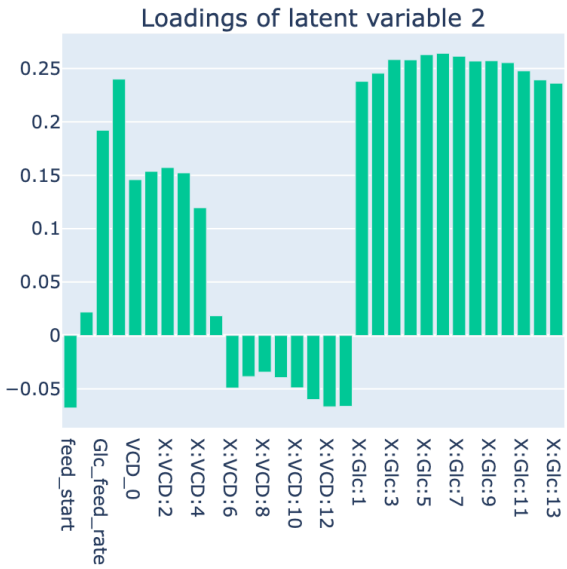
# Make predictions
yhat = pipe.predict(X)
X_test = pd.concat([doe_test, bwu_test.drop(set(remove_columns),axis=1)], axis=1)
y_test = tar_test
yhat_test = pipe.predict(X_test)
# Plot observed vs predicted with error metrics (R2, absolute and relative RMSE)
plothelpers.observed_vs_predicted(y, yhat, y_test, yhat_test)
```



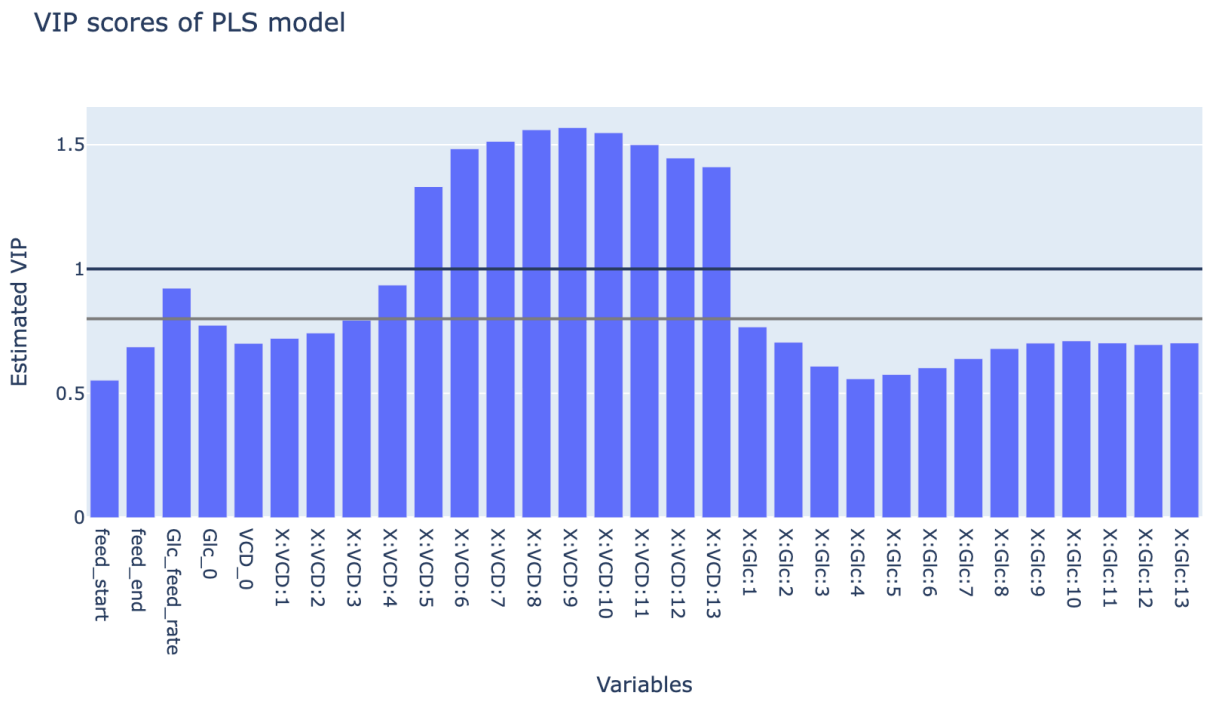

```
""" Latent variable on x-axis and y-axis """
LV_X_AXIS = 1
LV_Y_AXIS = 2
```








```
# VIP scores of the historical PLS model
pls_bwu_vip = modelhelpers.vip(X,y,pls_bwu)
# Plot VIP scores
fig = px.bar(x=list(X_columns),
             y=pls_bwu_vip.reshape(-1),
             title="VIP scores of PLS model",
             labels={'x':"Variables", 'y':"Estimated VIP",'color':"p-value"})
fig.add_hline(y=1)
fig.add_hline(y=0.8,line=dict(color='gray'))
fig.show()
```

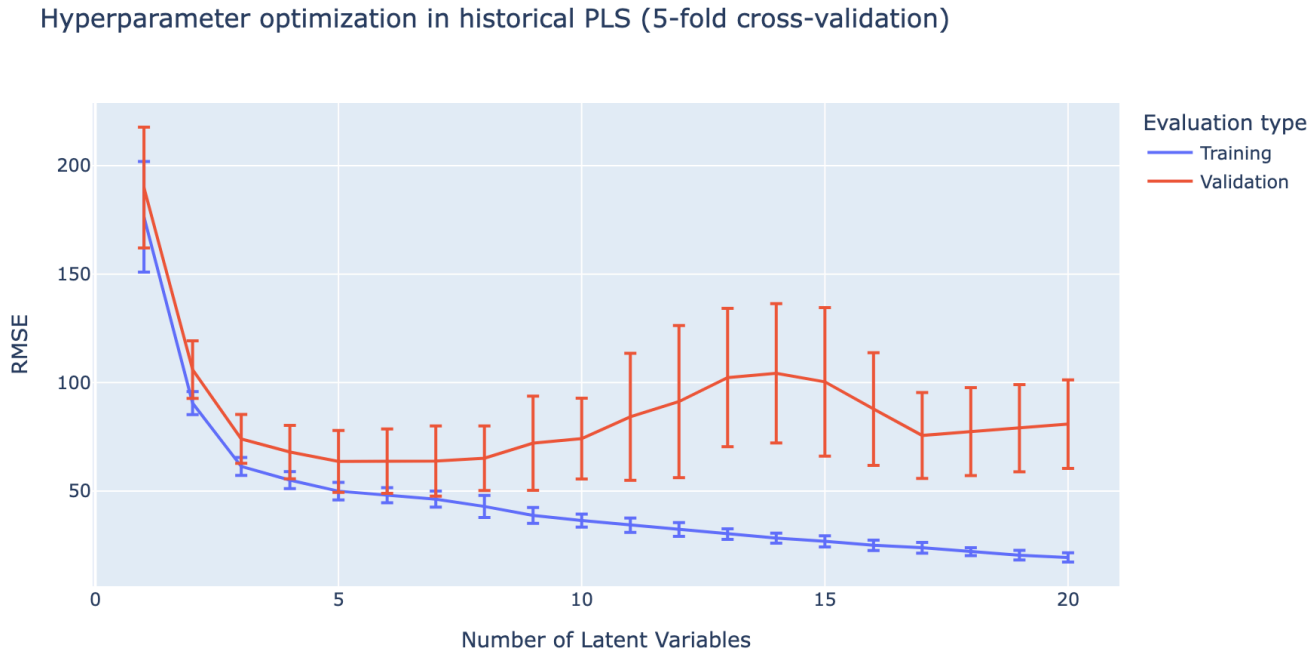


Hyperparameter optimization in cross-validation

Again we run a cross-validation to define the optimal number of latent variables of the historical model.

N-fold PLS models are trained using (N-1) folds each and evaluated on the remaining fold. This is repeated for different numbers of latent variables. The number of latent variables with the lowest validation RMSE is chosen as optimal. If two numbers of latent variables give a similar validation error, the smaller one is preferred, because it is more likely to produce stable predictions.

```
range_LV = range(1, min(X.shape[1], 20)+1)
train_eval, valid_eval = validation_curve(pipe, X, y, param_name =
"pls__n_components", param_range=list(range_LV),scoring =
'neg_root_mean_squared_error')
train_score=-np.mean(train_eval,axis=1)
valid_score=-np.mean(valid_eval,axis=1)
train_std = np.std(train_eval,axis=1)
valid_std = np.std(valid_eval,axis=1)
fig = go.Figure()
fig.add_trace(go.Scatter( x=list(range_LV), y=train_score,
error_y=dict(type='data',array=train_std,visible=True), name="Training"))
fig.add_trace(go.Scatter( x=list(range_LV), y=valid_score,
error_y=dict(type='data',array=valid_std,visible=True), name="Validation"))
fig.update_layout(title="Hyperparameter optimization in historical PLS (5-fold cross-
validation)",xaxis_title="Number of Latent
Variables",yaxis_title="RMSE",legend_title="Evaluation type")
fig.show()
```



Tasks

1. What is the optimal number of latent variables for the historical PLS model? Set `N_LV` to that value and compare the test set performance.
2. Check the model diagnostics of the historical model by plotting the residuals and verify whether the modelling assumptions are satisfied.
3. Reduce `USE_DAYS` to 7. How much does the prediction of the final titer deteriorate when only the first week of the process is known?

```
# Your code here
```